

# Chapter 7

## Normalization of Relational Tables

# Outline

- Modification anomalies
- Functional dependencies
- Major normal forms
- Relationship independence
- Practical concerns

# Modification Anomalies

- Unexpected side effect
- Insert, modify, and delete more data than desired
- Caused by excessive redundancies
- Strive for one fact in one place

# Big University Database Table

| <u>StdSSN</u> | StdClass | <u>OfferNo</u> | OffYear | EnrGrade | CourseNo | CrsDesc |
|---------------|----------|----------------|---------|----------|----------|---------|
| S1            | JUN      | O1             | 2006    | 3.5      | C1       | DB      |
| S1            | JUN      | O2             | 2006    | 3.3      | C2       | VB      |
| S2            | JUN      | O3             | 2006    | 3.1      | C3       | OO      |
| S2            | JUN      | O2             | 2006    | 3.4      | C2       | VB      |

# Modification Anomaly Examples

- Insertion
  - Insert more column data than desired
  - Must know student number and offering number to insert a new course
- Update
  - Change multiple rows to change one fact
  - Must change two rows to change student class of student S1
- Deletion
  - Deleting a row causes other facts to disappear
  - Deleting enrollment of student S2 in offering O3 causes loss of information about offering O3 and course C3

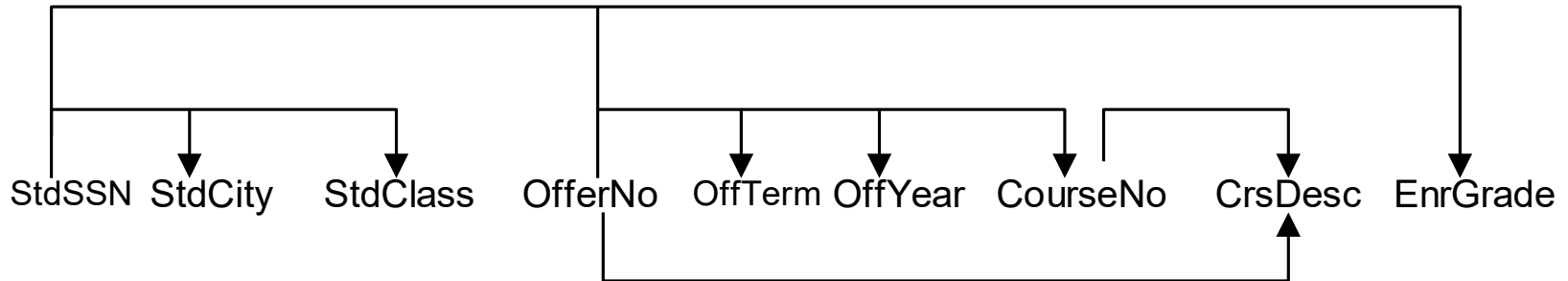
# Functional Dependencies

- Constraint on the possible rows in a table
- Value neutral like FKs and PKs
- Asserted
- Understand business rules

# FD Definition

- $X \rightarrow Y$
- X (functionally) determines Y
- X: left-hand-side (LHS) or determinant
- For each X value, there is at most one Y value
- Similar to candidate keys

# FD Diagrams and Lists



$\text{StdSSN} \rightarrow \text{StdCity}, \text{StdClass}$

$\text{OfferNo} \rightarrow \text{OffTerm}, \text{OffYear}, \text{CourseNo}, \text{CrsDesc}$

$\text{CourseNo} \rightarrow \text{CrsDesc}$

$\text{StdSSN}, \text{OfferNo} \rightarrow \text{EnrGrade}$

# FDs in Data

| <u>StdSSN</u> | StdClass | <u>OfferNo</u> | OffYear | EnrGrade | CourseNo | CrsDesc |
|---------------|----------|----------------|---------|----------|----------|---------|
| S1            | JUN      | O1             | 2006    | 3.5      | C1       | DB      |
| S1            | JUN      | O2             | 2006    | 3.3      | C2       | VB      |
| S2            | JUN      | O3             | 2006    | 3.1      | C3       | OO      |
| S2            | JUN      | O2             | 2006    | 3.4      | C2       | VB      |

- Prove non existence (but not existence) by looking at data
- Two rows that have the same X value but a different Y value

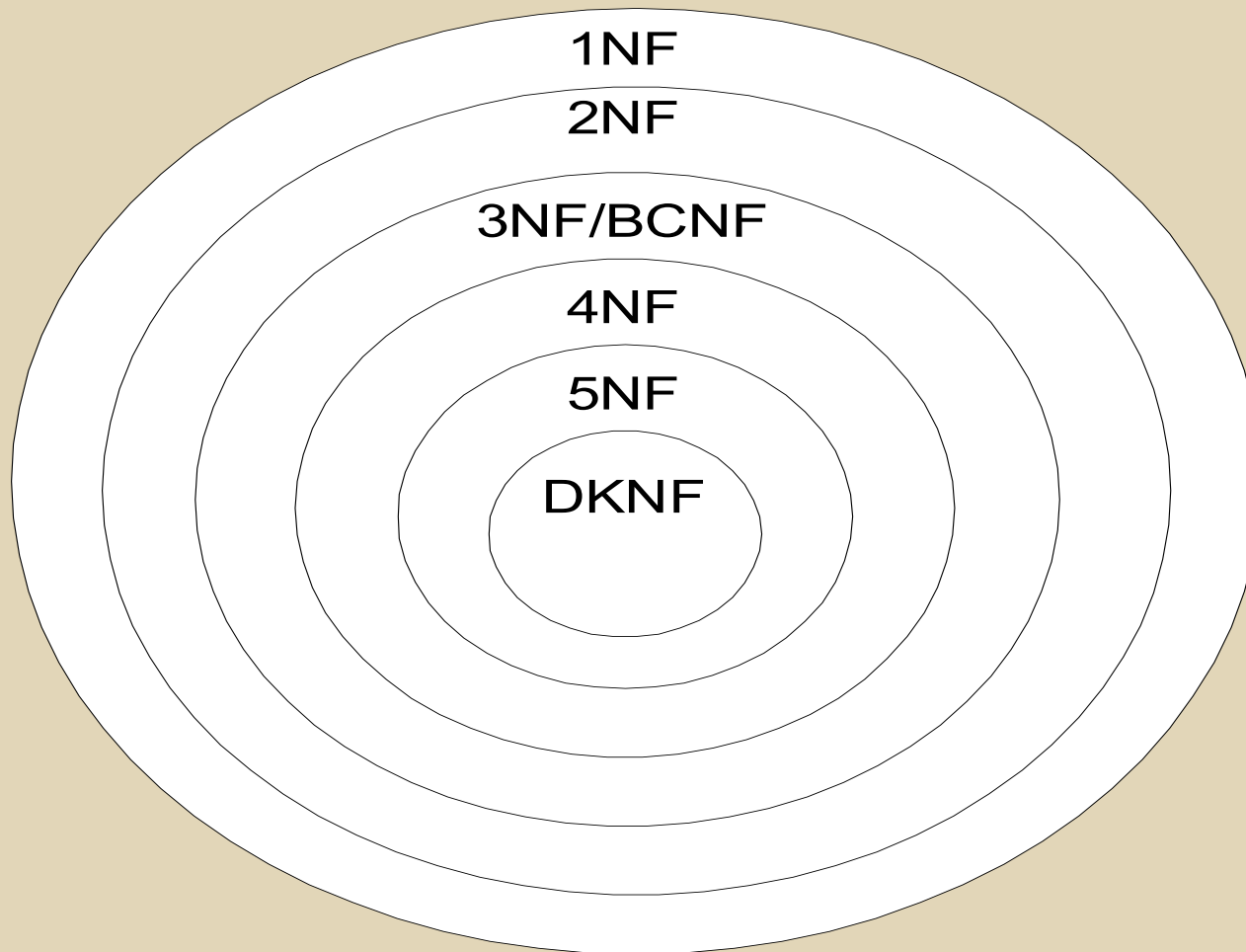
# Identifying FDs

- Easy identification
  - Statements about uniqueness
  - PKs and CKs resulting from ERD conversion
  - 1-M relationship: FD from child to parent
- Difficult identification
  - LHS is not a PK or CK in a converted table
  - LHS is part of a combined primary or candidate key
- Ensure minimality of LHS

# Normalization

- Process of removing unwanted redundancies
- Apply normal forms
  - Identify FDs
  - Determine whether FDs meet normal form
  - Split the table to meet the normal form if there is a violation

# Relationships of Normal Forms



# 1NF

- Starting point for most relational DBMSs
- No repeating groups: flat rows

| <u>StdSSN</u> | StdClass | <u>OfferNo</u> | OffYear | EnrGrade | CourseNo | CrsDesc |
|---------------|----------|----------------|---------|----------|----------|---------|
| S1            | JUN      | O1             | 2006    | 3.5      | C1       | DB      |
|               |          | O2             | 2006    | 3.3      | C2       | VB      |
| S2            | JUN      | O3             | 2006    | 3.1      | C3       | OO      |
|               |          | O2             | 2006    | 3.4      | C2       | VB      |

# Combined Definition of 2NF/3NF

- Key column: candidate key or part of candidate key
- Analogy to the traditional justice oath
- Every non key column depends on all candidate keys, whole candidate keys, and nothing but candidate keys
- Usually taught as separate definitions

# 2NF

- Every nonkey column depends on all candidate keys, not a subset of any candidate key
- Violations
  - Part of key → nonkey
  - Violations only for combined keys

# 2NF Example

- Many violations for the big university database table
  - StdSSN → StdCity, StdClass
  - OfferNo → OffTerm, OffYear, CourseNo, CrsDesc
- Splitting the table
  - **UnivTable1** (StdSSN, StdCity, StdClass)
  - **UnivTable2** (OfferNo, OffTerm, OffYear, CourseNo, CrsDesc)

# 3NF

- Every nonkey column depends only on candidate keys, not on non key columns
- Violations: Nonkey  $\rightarrow$  Nonkey
- Alternative formulation
  - No transitive FDs
  - $A \rightarrow B, B \rightarrow C$  then  $A \rightarrow C$
  - OfferNo  $\rightarrow$  CourseNo, CourseNo  $\rightarrow$  CrsDesc then OfferNo  $\rightarrow$  CrsDesc

# 3NF Example

- One violation in UnivTable2
  - CourseNo  $\rightarrow$  CrsDesc
- Splitting the table
  - **UnivTable2-1** (OfferNo, OffTerm, OffYear, CourseNo)
  - **UnivTable2-2** (CourseNo, CrsDesc)

# BCNF

- Every determinant must be a candidate key.
- Simpler definition
- Apply with simple synthesis procedure
- Special cases not covered by 3NF
  - Part of key  $\rightarrow$  Part of key
  - Nonkey  $\rightarrow$  Part of key
  - Special cases are not common

# BCNF Example

- Primary key: (OfferNo, StdSSN)
- Many violations for the big university database table
  - StdSSN  $\rightarrow$  StdCity, StdClass
  - OfferNo  $\rightarrow$  OffTerm, OffYear, CourseNo
  - CourseNo  $\rightarrow$  CrsDesc
- Split into four tables

# Simple Synthesis Procedure

1. Eliminate extraneous columns from the LHSs
2. Remove derived FDs
3. Arrange the FDs into groups with each group having the same determinant.
4. For each FD group, make a table with the determinant as the primary key.
5. Merge tables in which one table contains all columns of the other table.

# Simple Synthesis Example I

- Begin with FDs shown in Slide 8
- Step 1: no extraneous columns
- Step 2: eliminate OfferNo  $\rightarrow$  CrsDesc
- Step 3: already arranged by LHS
- Step 4: four tables (Student, Enrollment, Course, Offering)
- Step 5: no redundant tables

# Simple Synthesis Example II

- *AuthNo* → *AuthName*, *AuthEmail*, *AuthAddress*
- *AuthEmail* → *AuthNo*
- *PaperNo* → *Primary-AuthNo*, *Title*, *Abstract*, *Status*
- *RevNo* → *RevName*, *RevEmail*, *RevAddress*
- *RevEmail* → *RevNo*
- *RevNo*, *PaperNo* → *Auth-Comm*, *Prog-Comm*, *Date*, *Rating1*, *Rating2*, *Rating3*, *Rating4*, *Rating5*

# Simple Synthesis Example II

## Solution

- **Author**(AuthNo, AuthName, AuthEmail, AuthAddress)  
UNIQUE (AuthEmail)
- **Paper**(PaperNo, Primary-Auth, Title, Abstract, Status)  
FOREIGN KEY (Primary-Auth) REFERENCES Author
- **Reviewer**(RevNo, RevName, RevEmail, RevAddress)  
UNIQUE (RevEmail)
- **Review**(PaperNo, RevNo, Auth-Comm, Prog-Comm, Date, Rating1, Rating2, Rating3, Rating4, Rating5)  
FOREIGN KEY (PaperNo) REFERENCES Paper  
FOREIGN KEY (RevNo) REFERENCES Reviewer

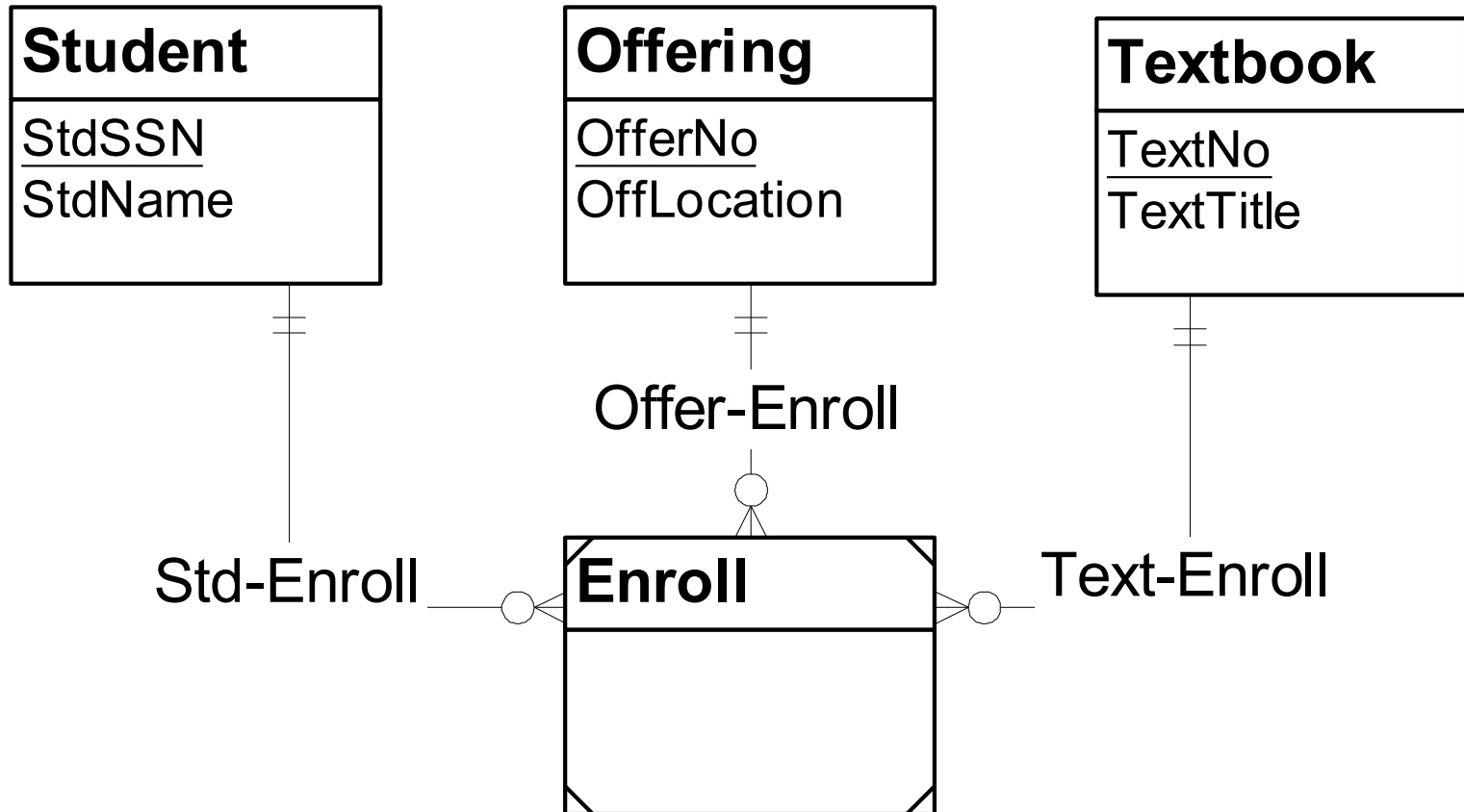
# Multiple Candidate Keys

- Multiple candidate keys do not violate either 3NF or BCNF
- Step 5 of the Simple Synthesis Procedure creates tables with multiple candidate keys.
- You should not split a table just because it contains multiple candidate keys.
- Splitting a table unnecessarily can slow query performance.

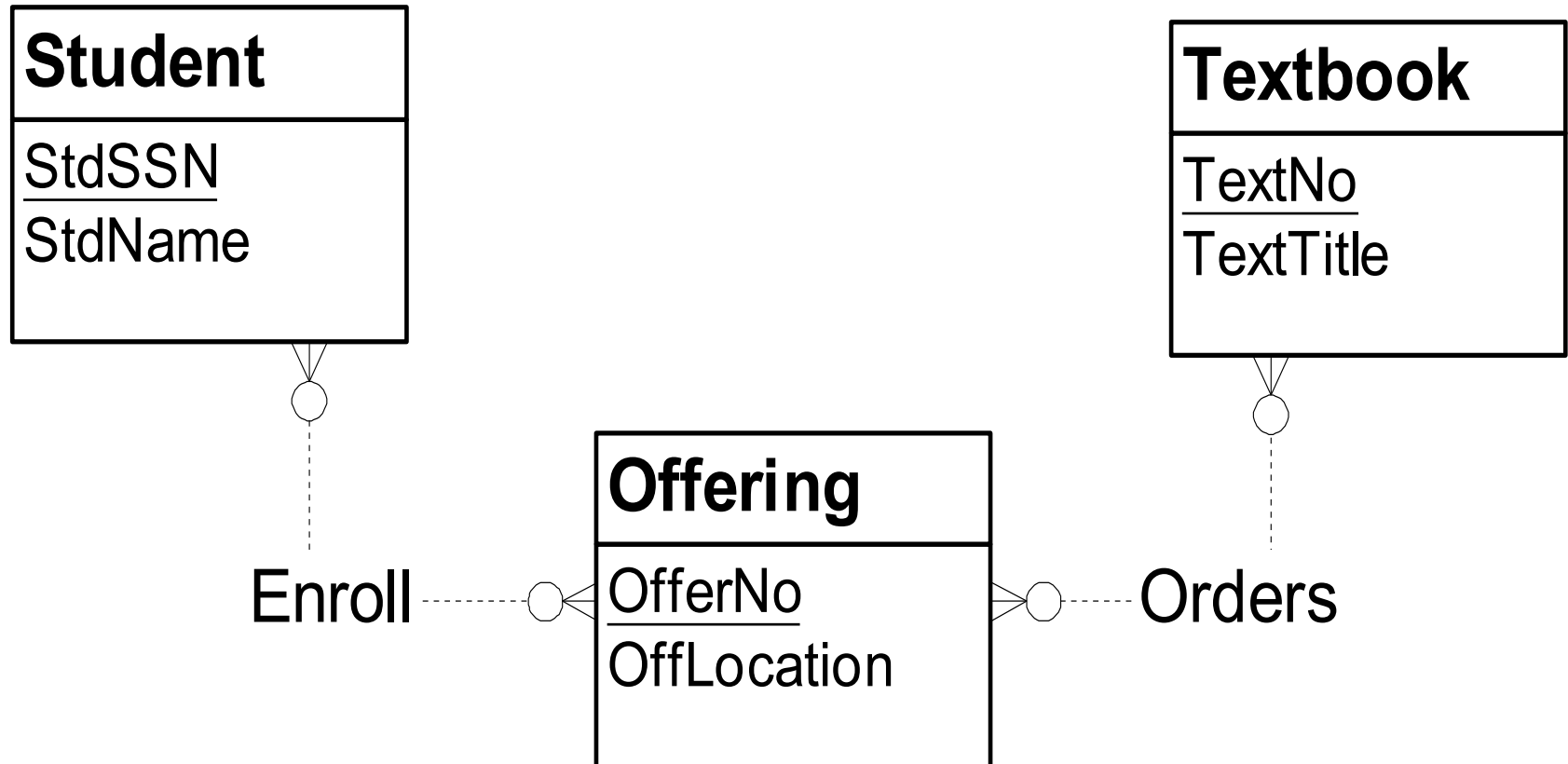
# Relationship Independence and 4NF

- M-way relationship that can be derived from binary relationships
- Split into binary relationships
- Specialized problem
- 4NF does not involve FDs

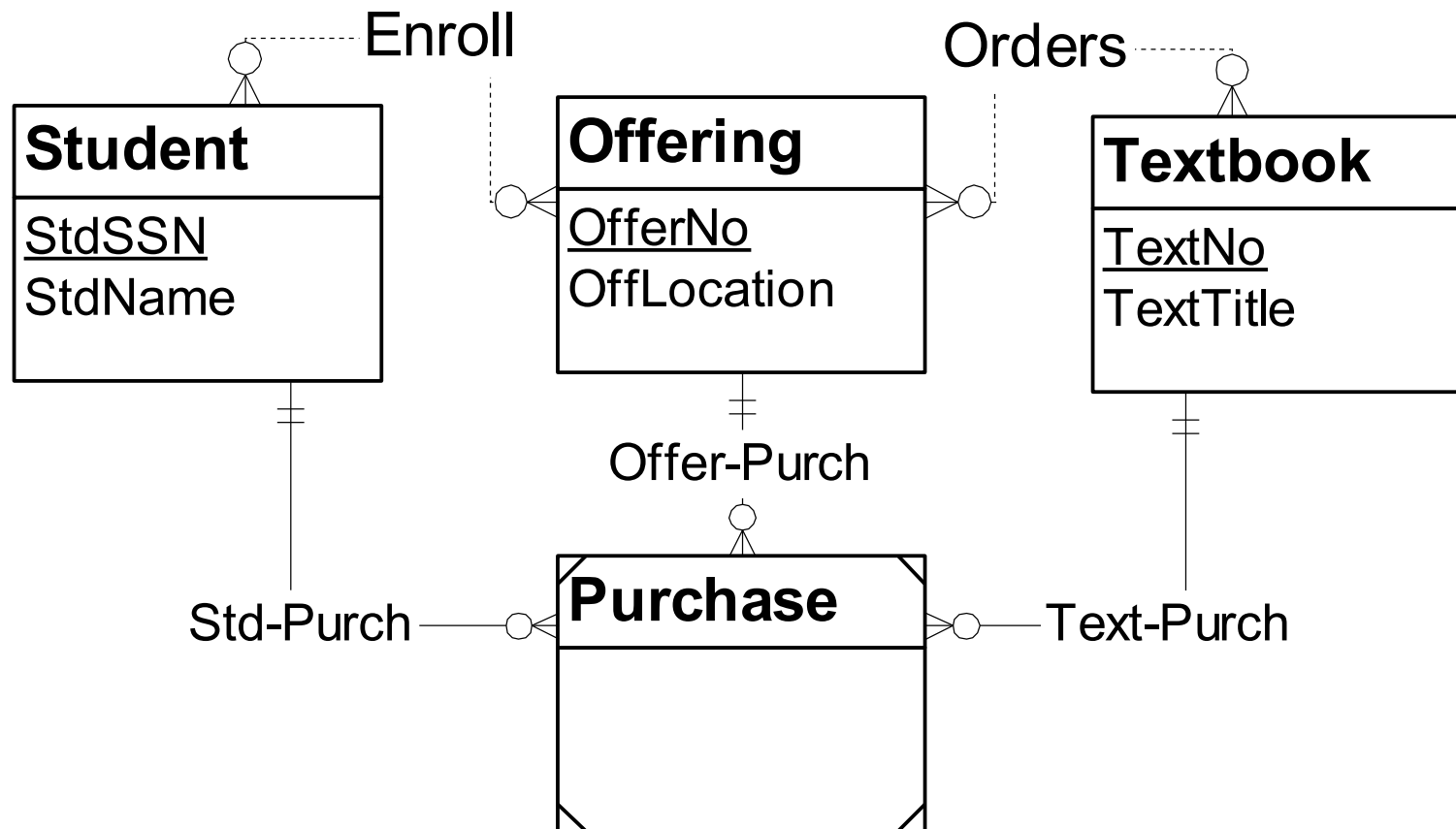
# Relationship Independence Problem



# Relationship Independence Solution



# Extension to the Relationship Independence Solution



# MVDs and 4NF

- MVD: difficult to identify
  - $A \twoheadrightarrow B \mid C$  (multi-determines)
  - A associated with a collection of B and C values
  - B and C are independent
  - Non trivial MVD: not also an FD
- 4NF: no non trivial MVDs

# MVD Representation

Given the two rows above the line, the two rows below the line are in the table if the MVD is true.

$$A \twoheadrightarrow B \mid C$$

| <u>A</u> | <u>B</u> | <u>C</u> |
|----------|----------|----------|
| A1       | B1       | C1       |
| A1       | B2       | C2       |
| A1       | B2       | C1       |
| A1       | B1       | C2       |

$$\text{OfferNo} \twoheadrightarrow \text{StdSSN} \mid \text{TextNo}$$

| <u>OfferNo</u> | <u>StdSSN</u> | <u>TextNo</u> |
|----------------|---------------|---------------|
| O1             | S1            | T1            |
| O1             | S2            | T2            |
| O1             | S2            | T1            |
| O1             | S1            | T2            |

# Higher Level Normal Forms

- 5NF for M-way relationships
- DKNF: absolute normal form
- DKNF is an ideal, not a practical normal form

# Role of Normalization

- Refinement
  - Use after ERD
  - Apply to table design or ERD
- Initial design
  - Record attributes and FDs
  - No initial ERD
  - May reverse engineer an ERD after normalization

# Advantages of Refinement Approach

- Easier to translate requirements into an ERD than list of FDs
- Fewer FDs to specify
- Fewer tables to split
- Easier to identify relationships especially M-N relationships without attributes

# Normalization Objective

- Update biased
- Not a concern for databases without updates (data warehouses)
- Denormalization
  - Purposeful violation of a normal form
  - Some FDs may not cause anomalies
  - May improve performance

# Summary

- Beware of unwanted redundancies
- FDs are important constraints
- Strive for BCNF
- Use a CASE tool for large problems
- Important tool of database development
- Focus on the normalization objective